

Fuzz Testing

Establish Settings Procedure

Revision

Version 3
6/23/26 6:55 PM

Author

Charles Wilson

Abstract

This document describes the procedure used to perform the establish settings activity described in the **AVCDL** ^[1] secondary document **Fuzz Testing Report** ^[2].

Group / Owner

Security / Security Architect

Motivation

This document is motivated by the need to have runtime-specific, cybersecurity-related feedback in the development of software for use within safety-critical, cyber-physical systems.

Note: Within the context of this document, the terms ***security*** and ***cybersecurity*** are used interchangeably. It is presumed that the term ***security*** is being used in reference to ***cybersecurity*** and not ***physical security***.

License

This work was created by **Torc Robotics** and is licensed under the **Creative Commons Attribution-Share Alike (CC BY-SA-4.0)** License.

<https://creativecommons.org/licenses/by/4.0/legalcode>

Audience

The audience of this document is the cybersecurity practitioner who will be conducting cybersecurity-specific fuzz testing.

Completeness of Output

Since fuzz testing is an as-is analysis, it is critical to ensure that the information gathered is as accurate and complete as possible. As with any cybersecurity assessment, it is not the place of the fuzz testing SME to make assumptions on the part of engineering. All information should be sourced from and confirmed by the owner of the element under consideration.

When information is not available for either a given section of the template or parts thereof, this should be noted. Major omissions should be recorded in the cybersecurity risk register.

Disposition of Output

Once completed, the generated output should be managed in the organization's document management system (DMS) as a document of record.

Entry Criteria

This document assumes that the reader understands the purpose of cybersecurity-relevant fuzz testing. Further, that the reader has read and understood the AVCDL **Fuzz Testing Report** ^[2] and **Secure Settings Document** ^[3] secondary documents.

Prerequisites – Development SME

Qualifications

It is required that the development SME is both a qualified and trained development SME with expertise in the element under consideration in the context of the tool under whose cybersecurity settings are being reviewed.

Note: The specific qualifications and training of development SMEs necessary to assert expertise in providing accurate information regarding the element under consideration is outside the scope of this document.

Note: The development group is responsible for determining and making available the appropriate SME for the analysis of the element under consideration.

Knowledge

It is required that the development SME have expertise in the element under consideration in the context of the tool under whose cybersecurity settings are being reviewed.

Prerequisites – Fuzz Testing SME

Qualifications

It is required that the fuzz testing SME is both a qualified and trained security architect (shown above on title page as **Owner**) as defined by the **NIST NCWF** role SP-ARC-002 and detailed in the **NCWF Roles - Security Architect** section of the **AVCDL Background Material** elaboration document ^[9].

Knowledge

It is required that the fuzz testing SME understands the purpose of cybersecurity-relevant tool settings.

Background Information

It is required that the fuzz testing SME has read and understands the AVCDL **Fuzz Testing Report**, and **Secure Settings Document** secondary documents. Additionally, that the fuzz testing SME has taken training relevant to this activity.

Prerequisites – DevOps SME

Qualifications

It is required that the DevOps SME is both a qualified and trained DevOps SME with expertise in the build system used for the element under consideration in the context of the tool whose cybersecurity settings are being modified.

Note: The specific qualifications and training of DevOps SMEs necessary to assert expertise in providing accurate information regarding the build system used for the element under consideration is outside the scope of this document.

Note: The DevOps group is responsible for determining and making available the appropriate SME for the modifications to build system needed for the tool under consideration.

Knowledge

It is required that the DevOps SME have expertise in the build system used for the element under consideration in the context of the tool under whose cybersecurity settings need modification.

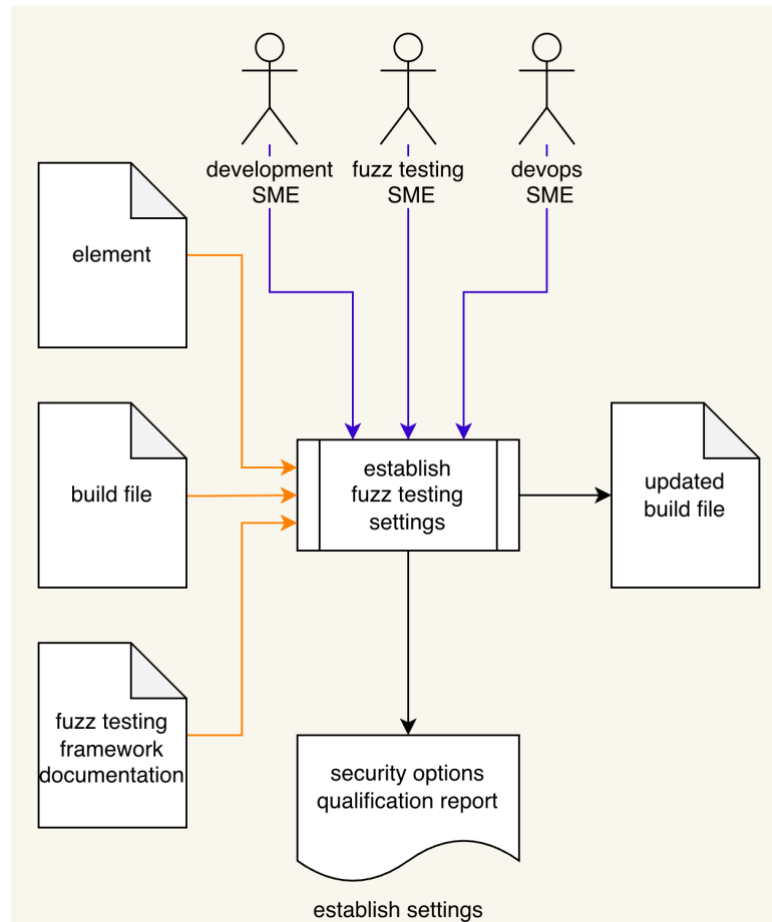
Prerequisites – Input Materials

It is required that the fuzz testing SME acquire all relevant documentation related to the tool under consideration necessary to complete the procedure.

It is required that the development group provide all relevant documentation related to the element under consideration necessary to complete the procedure.

Establish Settings Activity

The workflow diagram for the **establish settings** activity of the **Fuzz Testing Report** is shown below.



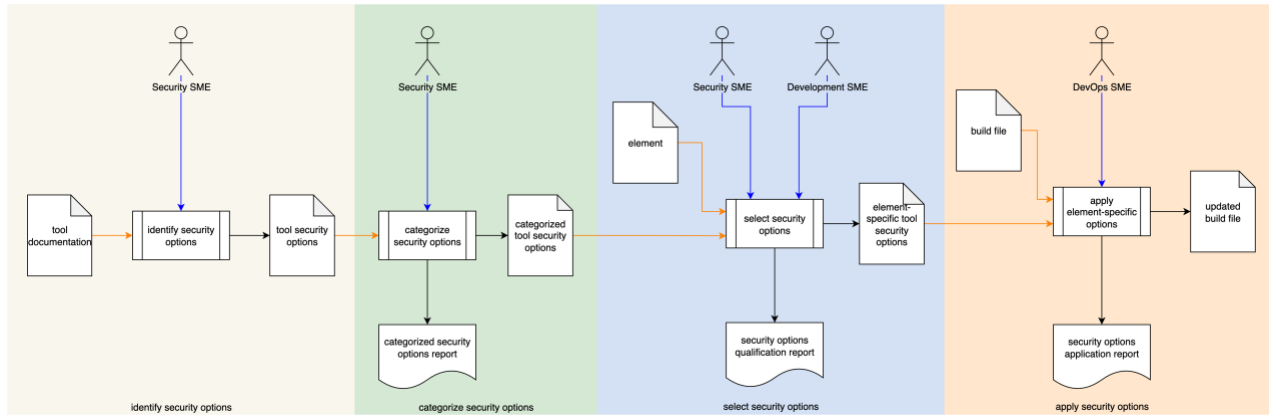
Using the **Element** under consideration, **Fuzz Testing Framework Documentation**, and **Build File**; the **Development SME**, **Fuzz Testing SME**, and **DevOps SME** work together following the **Secure Settings Document** ^[2] process to determine what element aspect needs to be tested and whether the element will require any kind of instrumentation to enable that testing. An **Updated Build File** will be produced. A **Security Options Qualifications Report** is generated.

Note: The scope of the element may be as small as a single file, or as large as the entire project.

Note: A link to a video ^[12] motivating the need to development team involvement is provided in the references section.

Settings Process Expanded

Consider the following diagram from the AVCDL **Secure Settings Document** secondary document.



Here we can see that the **establish settings** activity is more accurately described as a full process in and of itself. It is expressed as a single activity because its steps are common to multiple activities within the **AVCDL**.

Methodology

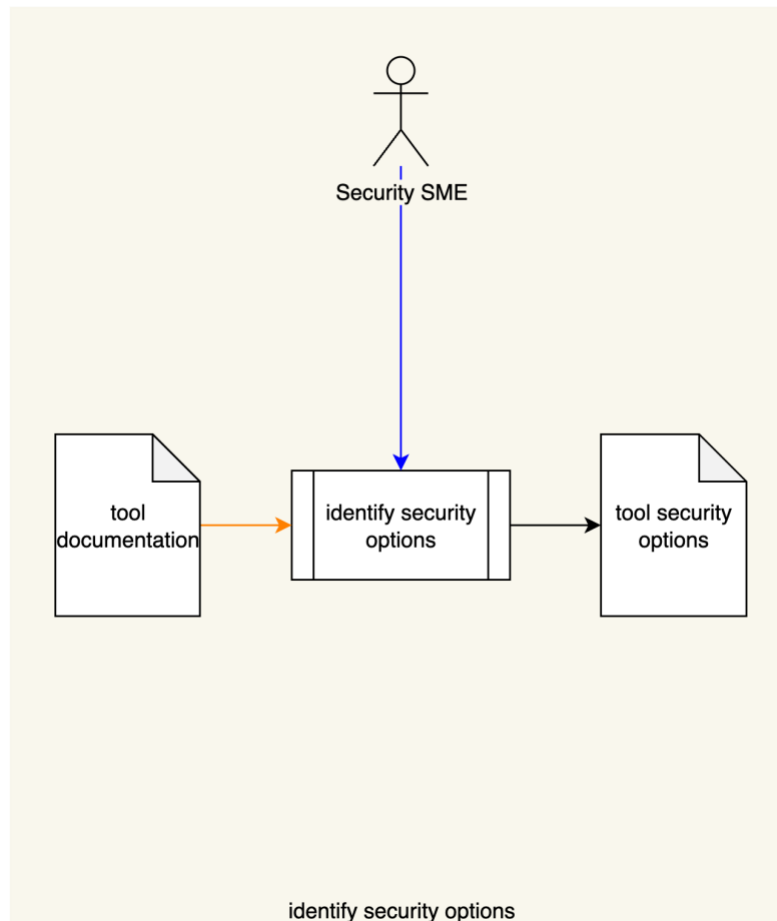
The methodology to use in this activity is to follow the procedures laid out in the **Secure Settings** procedures [\[4, 5, 6, 7\]](#).

Note: Because of the nature of fuzz testing, the activities laid out in the **Secure Settings** procedures will be deviated from. These deviations will be covered in the next section.

Discussion

The following provides fuzz testing-specific discussion regarding the application of the secure settings process. These will be addressed on an activity-by-activity basis.

Identify Options (identify frameworks)



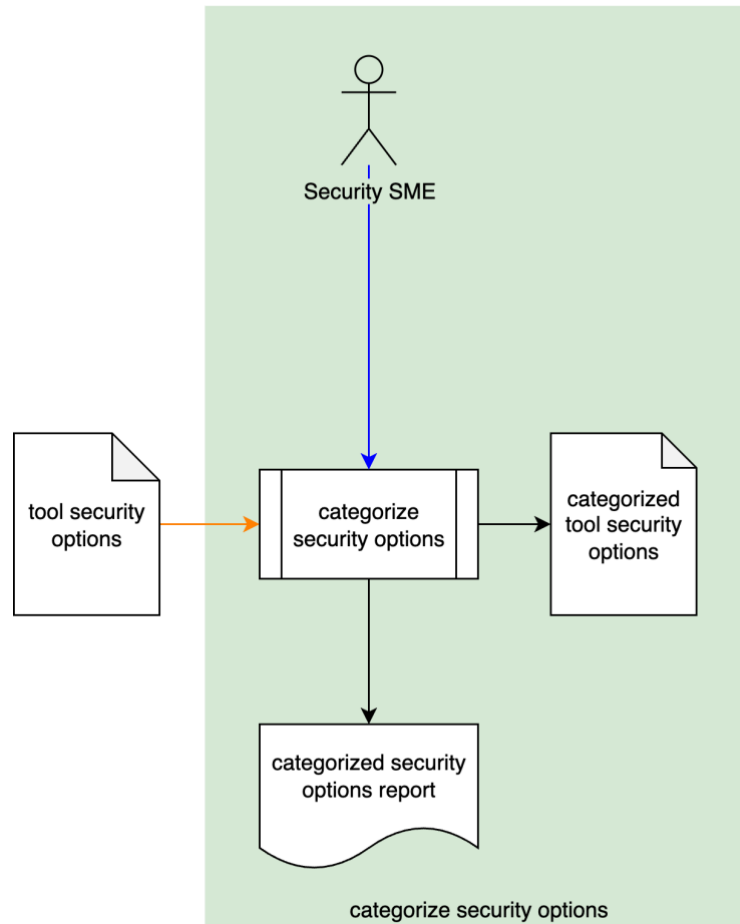
Within the context of fuzz testing, the goal of this activity is to identify the fuzz testing framework or frameworks that have application to the system under consideration. This differs from most of other approaches which focus on the element level. Here we are creating a catalog of frameworks from which we can select depending upon the specific build environment of individual elements.

Note: The information gathered in this activity may be reused for application to multiple elements.

Note: The general approach to this activity is covered in the **Secure Settings – Identify Options Procedure** AVCDL tertiary document.

Note: This activity uses a fuzz testing-specific version of the options template covered later in this document.

Categorize Options (determine use cases)



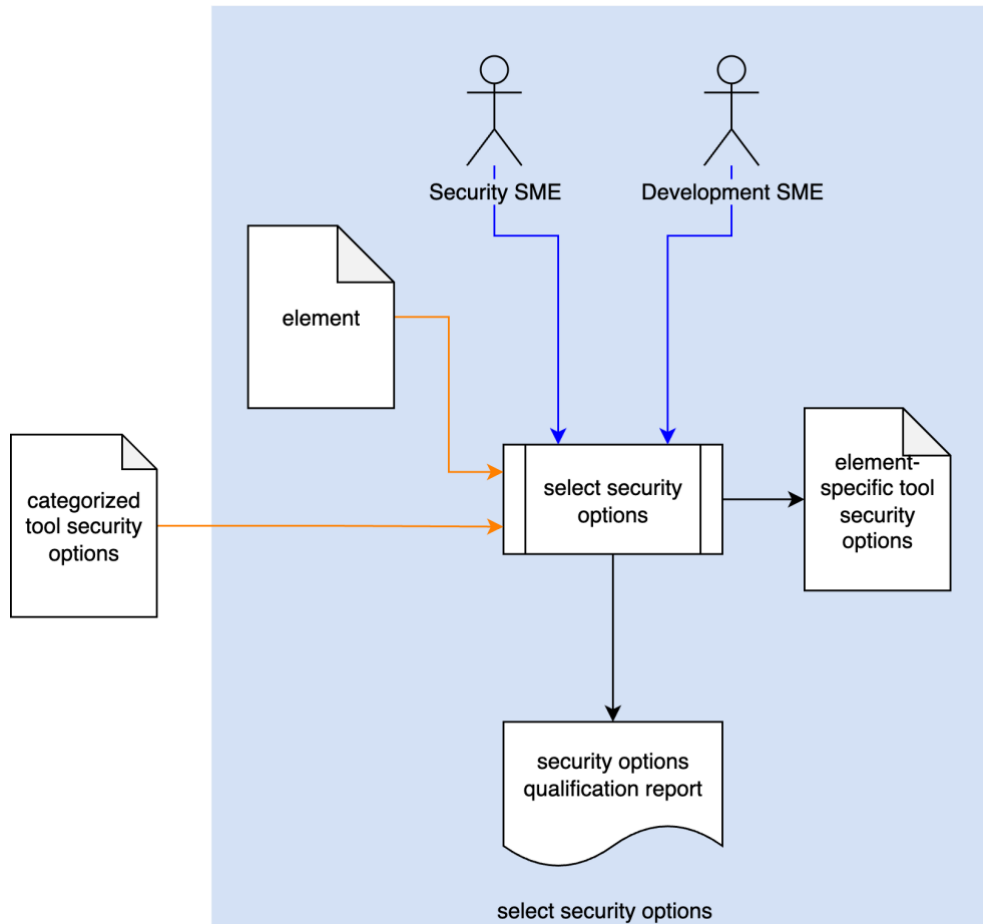
The goal of this activity is to establish specific use cases for the identified frameworks. Each use case (**category**) may be a particular configuration of operating system, build system, compiler, and fuzz testing framework features.

Note: The information gathered in this activity may be reused for application to multiple elements.

Note: The general approach to this activity is covered in the **Secure Settings – Categorize Options Procedure** AVCDL tertiary document.

Note: This activity uses a fuzz testing-specific version of the options template covered later in this document.

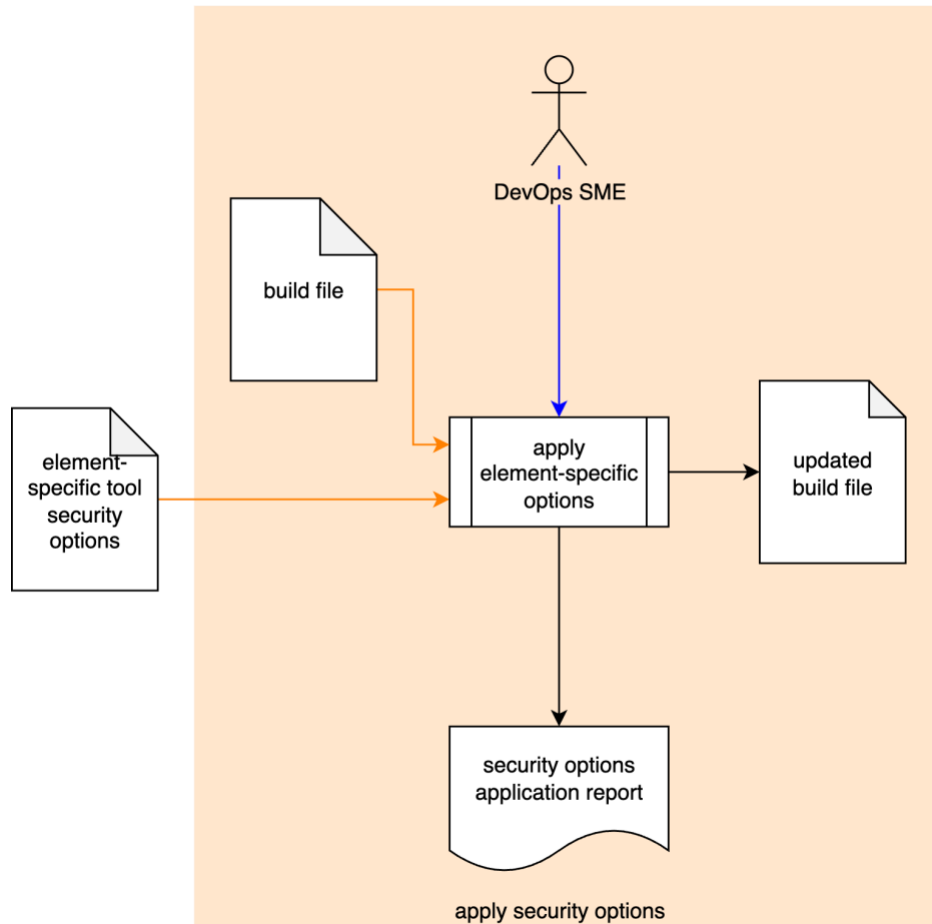
Select Options (map to element)



The goal of this activity is to identify which of the fuzz testing use cases (**categories**) are applicable for application to the element under consideration.

Note: The general approach to this activity is covered in the **Secure Settings – Select Options Procedure** AVCDL tertiary document.

Apply Options (adopt framework)



The goal of this activity is to emplace the selected fuzz testing frameworks and make any other changes required for them to be utilized by the fuzz tests created for the element.

Keep in mind that options are not always isolated from each other. This is especially true when dealing with compiler options. Verify that the selected options do not conflict with each other, and that they only operate within the desired scope.

Fuzz testing can be a resource intensive activity. Ensure that sufficient resources (time, memory, storage) will be available.

Note: It is recommended that when using fuzz testing frameworks for cybersecurity-specific fuzz testing within the build system that these be run separately in order to minimize the work required to perform the triage.

Note: The general approach to this activity is covered in the **Secure Settings – Apply Options Procedure** AVCDL tertiary document.

Fuzz Testing Settings Templates

The fuzz testing settings may be documented using the **AVCDL fuzz testing framework options template** (covered in the next section) and **AVCDL element-specific tool cybersecurity options template** Microsoft Excel workbooks [\[8, 11\]](#).

Note: The **AVCDL fuzz testing framework options template** is variant of the **AVCDL tool cybersecurity options template** Microsoft Excel workbooks [\[10\]](#). These workbooks are covered in the **AVCDL Secure Settings** tertiary documents [\[4, 5, 6\]](#).

Note: The reason for the use of a variant is that rather than individual tool options discussed in the Secure Settings process, the options referred to in this document are the various fuzz testing frameworks and their associated configurations.

Fuzz Testing Framework Options Template

The fuzz testing framework options may be documented using the **AVCDL fuzz testing framework options template** Microsoft Excel workbook [\[3\]](#).

Note: Other forms of documentation are permissible so long as they provide the information laid out in this document.

There are six sheets in the workbook. They are:

- [Cover sheet](#)
- [Revision history](#)
- [Reference documents](#)
- [Options](#)
- [Categories](#)
- [Legend](#)

These will be addressed in turn.

Duplication of Rows in the Various Sheets

When there is the need to add rows to the various sheets of the workbook, be sure to duplicate an existing row. This is because validation checks are attached to some of the cells which also enables the use of dropdown lists.

Cover Sheet

The **cover sheet** of the workbook is shown below:

Fuzz Testing Framework Options								
Cybersecurity SME	Cybersecurity SME							
Date	27-Aug-2000							
Revision	1							

Fields to be completed are shown in **red**.

Cybersecurity SME

This is the cybersecurity subject matter expert creating the fuzz testing framework options list.

Note: This is the **fuzz testing SME**. **Cybersecurity SME** is used here for consistency with the language used in the **Secure Settings** material.

Date

This is the date when the options list was created or updated. The date should be updated whenever the list is updated.

Revision

This is the revision number of the document. The revision number is a monotonic and increasing integer, starting at 1. It should be incremented every time the document is updated.

Revision History

The **revision history** sheet of the workbook is shown below:

Revision History		
Revision	Author	Description
1		initial revision

Revision

The **revision** corresponds to that listed on the cover sheet.

Author

The **author** corresponds to the cybersecurity SME listed on the cover sheet.

Description

This is a brief description of changes made to the workbook since it was last updated.

Reference Documents

Note: These references are those necessary for the creation of the fuzz testing framework options list.

Note: Every included framework should have at least one reference provided in this list.

The **references** sheet of the workbook is shown below:

Reference Documents		
Name	Description	Location

Name

This is the name of the document being referenced.

Description

This is a brief description of the document being referenced.

Note: In cases where the reference material is expressed in the form other than a document (for example, a web page), this should be noted.

Location

This is the location of the document being referenced. It may be a physical location or a URL.

Options

The **options** sheet of the workbook is shown below:

Options											
ID	option	framework	framework version	framework features	operating system	build system	compiler	minimum compiler version	compiler options	category	notes

ID

This is the unique ID of the option.

Note: An ID of the form

`FW_<FRAMEWORK>_<OS>_<build system>_<compiler>[-<zero-leading number>]`

is recommended (such as `FW_HF_LINUX_GCC` or `FW_FT_QNX_CLANG-01`).

The `<zero-leading number>` is intended for use when needed to provide additional granularity, such as when different versions of a compiler require different compiler options (`FW_HFP_LINUX_GCC-01` and `FW_HFP_LINUX_GCC-02`).

Option

This is the short name assigned to this particular framework configuration.

Note: It may reflect the ID (`honggfuzz_persistent_clang`) or be generic when only one variant exists (`FuzzTest`).

Framework

This is the name of the framework.

Note: This is a text field. If there is a common set of values, the field may be converted to a list populated from a column on the **legend** sheet.

Framework Version

This is the version of the framework.

Framework Features

This is a list of framework features needed for this option.

Operating System

This is the operating system the framework is used on.

Note: This is a list field. Common values have been provided for the list. The list is populated from a column on the **legend** sheet.

When additional values are added to the list, the template version (located at the bottom of the **legend** sheet) should be updated.

Build System

This is the build system the framework works with.

Note: This is a list field. Common values have been provided for the list. The list is populated from a column on the **legend** sheet.

When additional values are added to the list, the template version (located at the bottom of the **legend** sheet) should be updated.

Compiler

This is the compiler the framework uses.

Note: This is a list field. Common values have been provided for the list. The list is populated from a column on the **legend** sheet.

When additional values are added to the list, the template version (located at the bottom of the **legend** sheet) should be updated.

Minimum Compiler Version

This is the minimum compiler version required for this option.

Compiler Options

This is the list of compiler options need to enable the framework.

Category

This is the use case for this framework option.

Note: This field is an auto-generated list populated from the **categories** sheet.

Notes

This is a general notes field.

Categories

Note: The general approach to the categories sheet is covered in the **Secure Settings – Categorize Options Procedure** AVCDL tertiary document.

In the context of fuzz testing a **category** is a **use case**.

The **categories** sheet of the workbook is shown below:

Categories				
ID	category	description	qualifications	notes

ID

This is the unique ID of the category.

Note: An ID of the form **CAT_<CATEGORY_CLASSIFIER>-<zero-leading number>** is recommended (such as **CAT_FFW-01**).

Category

This is the name of the category.

Note: This column is used to populate the **category** field list on the **options** sheet.

It is recommended that these names be clear and concise (such as **general - source** or **UDS - binary**).

Description

This is the description of the category.

Qualifications

This field describes why an option would be placed in this category.

Notes

This is a general notes field.

Legend

The **legend** sheet of the workbook is shown below:

operating system	build system	compiler
Linux	Bazel	clang
MacOS	CMake	gcc
QNX	make	N/A
Windows	N/A	
N/A		

The **legend** sheet information is used to make the completion of the document easier by providing dropdown lists for common values. It also ensures that spelling errors do not creep into the generated material.

Note: The legend sheet of an instance of the template should not be edited. If an unlisted value is required, the template should be separately revised.

Remember to increment the revision number at the bottom of the template page when making revisions.

Exit Criteria

This procedure is considered complete once the generated output has been entered into the organization's DMS as a document of record.

Note: The processes and procedures for entering documents into the DMS, or the updating thereof, are outside the scope of this document.

References

1. **AVCDL** (AVCDL primary document)
2. **Fuzz Testing Report** (AVCDL secondary document)
3. **Secure Settings Document** (AVCDL secondary document)
4. **Secure Settings – Identify Options Procedure** (AVCDL tertiary document)
5. **Secure Settings – Categorize Options Procedure** (AVCDL tertiary document)
6. **Secure Settings – Select Options Procedure** (AVCDL tertiary document)
7. **Secure Settings – Apply Options Procedure** (AVCDL tertiary document)
8. **AVCDL fuzz testing framework options template** (AVCDL template)
9. **AVCDL Background Material** (AVCDL elaboration document)
10. **AVCDL tool cybersecurity options template** (AVCDL template)
11. **AVCDL element-specific tool cybersecurity options template** (AVCDL template)
12. **Christian Holler: The Human Component in Bug Finding @FuzzCon Europe 2020**
<https://www.youtube.com/watch?v=ifc2C5fLIWU>